Activity 4 Pthread programming

<u>วัตถุประสงค์</u>

1. เพื่อให้นิสิตสามารถเขียนโปรแกรม pthread แบบง่ายๆ ได้
2. เพื่อให้นิสิตเข้าใจความสัมพันธ์ระหว่าง process และ thread

<u>กิจกรรมในชั้นเรียน</u>

1. ขอให้ศึกษาโปรแกรม thread ข้างล่าง

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>

void* say_hello(void* data)
{
      char *str;
      str = (char*)data;
      while(1)
      {
            printf("%s\n",str);
            sleep(1);
      }
}

void main()
{
      pthread_t t1,t2;

      pthread_create(&t1,NULL,say_hello,"hello from 1");
      pthread_create(&t2,NULL,say_hello,"hello from 2");
      pthread_join(t1,NULL);
      pthread_join(t2,NULL);

}
```

จากนั้นดัดแปลงโปรแกรมนี้ ให้สร้าง 2 thread ที่รับ argument 2 ตัว

โดยที่ thread ตัวแรกแสดง argument แรก

thread ตัวที่สองแสดง argument ตัวที่สอง

ตัวอย่างผลลัพธ์

สมมติว่า โปรแกรมชื่อ thread_sol1 เมื่อทำงานแล้ว จะได้ผลลัพธ์ดังนี้

```
Thong@Debian >thread_sol1
usage thread_sol1 arg1 arg2
Thong@Debian >
Thong@Debian >
Thong@Debian >thread_sol1 cedt chula
This is thread cedt
This is thread chula
This is thread cedt
This is thread chula
This is thread cedt
This is thread chula
This is thread cedt
This is thread chula
This is thread chula
This is thread cedt
^C
Thong@Debian >
```

ถ้าผู้ใช้ใส่ argument **ไม่ครบสองตัว โปรแกรมจะต้องขึ้นข้อความว่า**

usage : thread_sol1 arg1 arg2

เปิดอีกหน้าต่าง แล้วป้อนคำสั่ง *ps m -o pid,tid,command*

แล้ว capture หน้าจอ แสดงให้เห็นว่า มี thread เกิดขึ้น

```
Thong@Debian >ps m -o pid,tid,command
  PID   TID COMMAND
    9     - -bash
    -     9 -
 8681     - -bash
    -  8681 -
28013     - thread_sol1 cedt chula
    - 28013 -
    - 28014 -
    - 28015 -
28017     - ps m -o pid,tid,command
    - 28017 -
Thong@Debian >
```

2. จงเขียนโปรแกรมที่โปรเซสแม่สร้างโปรเซสลูก โดยที่

- โปรเซสแม่สร้าง thread 2 ตัว

  โดยที่ thread ตัวแรกขึ้นข้อความ “First thread from parent process”

  thread ตัวที่สองขึ้นข้อความ “Second thread from parent process”

- โปรเซสลูกสร้าง thread 2 ตัว

  โดยที่ thread ตัวแรกขึ้นข้อความ “First thread from child process”

  thread ตัวที่สองขึ้นข้อความ “Second thread from child process”

ตัวอย่างผลลัพธ์

ถ้านิสิต run โปรแกรมหลายครั้ง จะเห็นว่าโปรเซสแม่กับลูกจะแย่งกัน

บางครั้งลูกมาก่อน บางครั้งแม่มาก่อน ไม่มีความแน่นอน

```
Thong@Debian >thread_proc
First thread from parent process
First thread from child process
Second thread from child process
Second thread from parent process
Thong@Debian >!!
thread_proc
Second thread from child process
First thread from parent process
First thread from child process
Second thread from parent process
Thong@Debian >!!
thread_proc
First thread from parent process
Second thread from parent process
Second thread from child process
First thread from child process
Thong@Debian >
```

การส่งงาน

ส่งเป็นไฟล์ PDF เพียงไฟล์เดียวที่มี source code และผลลัพธ์ที่ capture จากหน้าจอทั้ง 2 ข้อ โดยใส่ชื่อสมาชิกทุกคนด้วย